

## **Teória [str. 2 - 10]**

- 1. Problematika prepisu dát v SSD**
- 2. Maximálny počet výpadkov v N-úrovňových tabuľkách stránok**
- 3. Možný počet úrovní hierarchií tabuliek stránok**
- 4. Plánovanie prístupu na disk**
- 5. Maximálny počet prístupov do RAM v N-úrov. Tab. str.**
- 6. Počet blokov súboru o veľkosti 360.5kB . Alokačný blok o veľkosti 1kB, odkaz 10B, 60.5kB v UFS**
- 7. Completely Fair Scheduler (CFS)**
- 8. Akcie OS pri operácii close()**
- 9. Účel a princíp dvoj-úrovňovej obsluhy prerušenia**
- 10. Akcia pri výpadku str. (FIFO, LRU)**
- 11. Plánovanie**
- 12. Kroky systému pri volaní open()**
- 13. Deadlock**
- 14. Operačný Systém, Jadro, Mikrojadro**
- 15. Žurnálovanie**
- 16. Súbor popísaný B+ stromom**
- 17. Alokačný blok, Diskový sektor, Extent**
- 18. Preklad logickej adresy na fyzickú cez 2-úrovňovú tabuľku stránok**
- 19. Inverzia priorít**
- 20. Nežiadúce účinky využitia operácie spinlock**
- 21. NTFS, odlišnosti rôzne dlhých súborov, 3 varianty reprezentácie a adresovania**

## **Programy [str. 10 - 13]**

## **Vypracované zadania [str. 13 - 17]**

### 1. Problematika prepisu dát v SSD [pr 4 , sl 8]

- Prázdné stránky ide zapisovať jednotlivo. Pre prepis je nutné načítať celý blok do vyrovnávacej pamäte, v nej zmeniť, na disku vymazať a spätne zapísať.
- Riešenie, teda minimalizácia problému:
  - o SSD môže mať viac stránok, než je oficiálna kapacita
  - o Príkaz TRIM- umožňuje súborovému systému zdeliť SSD, ktoré stránky nie sú používané a následne vymazať bloky tvorené takými stránkami. (Nie vždy sa dá použiť.)
  - o Radič SSD môže tiež sám interne presúvať stránky, aby mohol niektoré bloky uvoľniť

### 2. Maximálny počet výpadkov v N-úrovňových tabuľkách stránok [pr 7, sl 26]

- K výpadkom môže dôjsť v dátach aj v inštrukciách
- Inštrukcie môžu zasahovať do dvoch rôznych stránok
- Každá z uvedených stránok môže byť prekladaná cez rôzne dielčie tabuľkové stránky (s výnimkou dielčích tab. str. najvyššej úrovne.) , pretože popis stránok, do ktorých zasahuje inštrukcia, môže byť na rozmedzí tabuľkových str.
- Každá dielčia tabuľková stránka (okrem najvyššej úrovne) môže vypadnúť. Vypadnúť ich teda môže N (alebo napr. N-1 bez najvyššej úrovne) pre každú sprístupnenú stránku.
- 1 na začiatku – prvé sa preložia stránky, až potom prvá, druhá, atď. úroveň

### 3. Možný počet úrovní hierarchií tabuliek stránok [pr 7, sl 16,17]

- Pre posuv v stránke treba 32b
- $128 - 32 = 96b$  ostávajúcich pre stránkovanie
- $128b = 2^4B$  a teda dielčie tab. str. majú  $2^{32} / 2^4 = 2^{28}$  riadkov
- Pre plné využitie dielčích tabuľkových str. je treba 28b adresy
- $96/28 = 3.42 \rightarrow$  len 3 odkazy
- Do 96b sa tým pádom vojdú, pri danej tolerancii nepoužitých bitov adresy, 3 odkazy tab. str. a dostávame teda 3-úrovňové tab. str.

### 4. Plánovanie prístupu na disk [pr 4, sl 14]

- Prichádzajúce I/O požiadavky sú ukladané do vyrovnávacej pamäte a ich poradie sa prípadne mení s cieľom minimalizovať réžiu diskových operácií
- Výťahový algoritmus: pohybuje hlavičkami od stredu k okraji platní a späť, požiadavky na I/O vybavuje v poradí zodpovedajúcim pohybom hlavičiek
- Ďalšie plánovacie algoritmy:
  - o Circular SCAN
  - o LOOK, C-LOOK (NOOP,...)
  - o združovanie operácií
  - o vyvažovanie operácií rôznych procesov
  - o priority operácií
  - o odkladanie operácií
  - o časové limity operácií

### 5. Maximálny počet prístupov do RAM v N-úrov. Tab. str. [pr7, sl 13,16,17]

- $4N + 4 = 4*(N+1)$
- Prístup pre dáta i kód inštrukcie
- Oboje prípadne nezarovnané – 4 stránky (pokiaľ budú stránky zarovnané, môžu byť len 2)
- Pre každú stránku preklad cez N-úrovní

**6. Počet blokov súboru o veľkosti 360.5kB . Alokačný blok o veľkosti 1kB, odkaz 10B, 60.5kB v UFS**

- Spolu 366 alokovaných blokov
  - o 361 Dátových blokov
  - o 1 s priamymi odkazmi dostupnými z i-uzlu cez nepriame odkazy
  - o 1 s nepriamymi odkazmi prvej úrovne a z neho 3 s priamymi odkazmi dostupnými z vyššie uvedeného bloku

**7. Completely Fair Scheduler (CFS) [pr 5, sl 24]**

- Explicitne sa snaží poskytnúť procesom odpovedajúce % strojového času podľa priorít
- Vedie si údaj o spotrebovanom čase procesov (ich virtuálny procesorový čas)
- Vedie si údaj o minimálnom spotrebovanom čase, pre novo pripravené procesy
- Vyberá proces s najmenším stráveným časom
- Procesy necháva bežať po nejakej časovej kvantume s rýchlosťou virtuálneho času ovplyvnené prioritou. (nižšej prioritne beží čas rýchlejšie)
- Podporuje skupinové plánovanie – rozdeľuje čas spravodlivo pre procesy spustené z iných terminálov

**8. Akcie OS pri operácii close() [pr4 , sl 60]**

- Kontrola platnosti fd
- Uvoľní položku v tabuľke deskriptorov, zníži počítadlo odkazov v tabuľke otvorených súborov
- Pokiaľ je počítadlo odkazov nulové, uvoľní položku, zníži počet odkazov vo v-uzle. Potom i-uzol sa z v-uzlu odkopíruje do VP (virtuálnej pamäte) a uvoľní
- Vracia 0, alebo pri chybe -1

**9. Účel a princíp dvoj-úrovňovej obsluhy prerušenia [pr 2, sl 14]**

- Minimalizácia dialky zákazu prerušenia
- 1. Úroveň
  - o zaisťuje minimálnu obsluhu HW, plánovanie behu 2. Úrovne
  - o môže byť vylúčené/zamaskované ďalšie prerušenie rovnakého typu
- 2. Úroveň
  - o postupne dokončuje obsluhu zaznamenaných prerušení
  - o nemusí zakazovať prerušenie – vzájomné vylúčenie sa dá riešiť bežnými synchronizačnými prostriedkami
  - o obsluha môže bežať vo špeciálnych procesoch (interrupt threads vo FreeBSD, tasklety/softIRQ v Linuxe)
  - o vzájomné vylúčenie sa dá riešiť bežnými synchronizačnými prostriedkami (semaforey a podobne)
  - o Môžu tu byť samostatné úrovne priorít

**10. Akcia pri výpadku str. [pr 7 , sl 24-27]**

- Kontrola, či proces neodkazuje mimo pridelený adresový priestor
- Alokácia rámca, použijeme voľný rámec, ak nejaký je
  - o ak nie je, vyberieme vhodnú stránku s prideleným rámcom (victim page)
  - o tu odložíme do swap (ak je nutné: príznak modifikácie v tabuľke stránok)
  - o použijeme voľný rámec
- Inicializácia stránky po alokácii je závislá na predchádzajúcom stave stránky
  - o prvý odkaz na stránku:
    - Z programu načítame kód a inicializované dáta
    - Všetko ostatné vynulujeme (nie je možné ponechať pôvodný obsah z bezpečnostných dôvodov)
    -

- stránka bola v minulosti uvoľnená z FAP
  - Kód a konštantné dáta znovu načítame z programu
  - Ostatné: ak bola stránka modifikovaná je v swape a musí sa načítať do FAP
- Úprava tab. str. (namapovanie zprístupnených strániek na prideľný rámec)
- Restart inštrukcie (proces je v stave pripravený)

Pri výpadku stránky môže dôjsť k odložení stránky. Algoritmy výberu odkladania stránok:

### Least Recently Used (LRU)

- Odkladá najďalej nepoužitú stránku
- Veľmi dobrá aproximácia hypotetického ideálneho algoritmu (poznal by budúcnosť a podľa budúcich požiadavkou by sa rozhodoval, čo odložiť.)
- Problematická implementácia -> vyžaduje výraznú HW podporu (označovanie stránok časovým razítkom posledného prístupu či udržiavanie zásobníku stránok)
- Používajú sa aproximácie LRU
- Aproximácie LRU pomocou **obmedzenej histórie**
  - pri každom prístupe HW nastaví referenčný bit stránky
  - jadro si vedie obmedzenú históriu tohto bitu pre stránky
  - periodicky posúva obsah doprava -> na pozíciu úplne vľavo (na začiatok) uloží aktuálnu hodnotu referenčného bitu a vynuluje ho
  - obeť (victim page) je vybraná ako stránka s najnižšou číselnou hodnotou histórie
- Aproximácie LRU algoritmom **druhé šance**
  - Stránky sú v kruhovom zozname
  - Postupujem a nulujeme ref. bit
  - Odstránime prvú stránku, ktorý už nulový referenčný bit má
  - často používaný algoritmus (clock algorithm)
- Výhody LRU: často používaný, veľmi dobrá aproximácia
- Nevýhody LRU: problematická implementácia (vyžaduje výraznú HW podporu)

### First In, First Out (FIFO)

- Jednoduchá implementácia
- Môže odstrániť „starú“, ale často používanú stránku
- Trpí tzv. Beladyho anomáliou
- Odstraňuje stránku, ktorá bola zavedená do pamäti pred najdlhšou dobou a doposiaľ nebola odstránená

## 11. Plánovanie

- Preemptívne: ku zmene bežiaceho procesu môže dôjsť na základe prerušenia (typicky od časovača, ale môže sa jednať aj o iné prerušenie)
- Nepreemptívni: ku zmene bežiaceho procesu môže dôjsť iba vtedy, keď to bežiaci proces umožní, predaním riadenia jadru. (I/O operácie, alebo sa sám vzdá procesoru – volanie yield)
- **FCFS (First Come, First Served) – NEPREEMPT, FIFO.** Pri vzniku procesu, jeho uvoľnenie z čakania, alebo vzdialenia sa procesu od procesoru, je tento proces zaradený na koniec fronty. Procesor sa prideľuje prvému procesu vo fronte.
- **Round-robin – PREEMPT, FIFO.** Obdoba FCFS, najviac má ale každý proces pridelené časové kvantum, po vypršaní tohto kvanta je mu odobraný procesor zaradený na koniec radu. Procesor sa prideľuje prvému procesu vo fronte.

- **SJF (Shortest Job First)** – *NEPREEMPT*, *prideluje podľa CPU burst*. Minimalizuje priemernú čakaciu dobu, zvyšuje priepustnosť systému. Nutnosť rozumné odhadnúť dobu behu procesu. Používa sa pre opakovane robené podobné úlohy, najmä v špecializovaných dávkových systémoch.
- **SRT (Shortest Reming Time)** – obdoba SJF s *PREEMPCIOU* pri vzniku či uvoľnení procesu.
- **Viacúrovňové plánovanie**: procesy sú rozdelené do skupín (typicky podľa priority), v každej skupine môže byť iný plánovací algoritmus (FCFS,...), ďalšie plánovací algoritmus sa používa typicky pre určenie skupiny, z ktorej sa vyberie proces, ktorý má bežať (ale typicky na základe priority), môže hroziť hladovanie.
- **Viacúrovňové plánovanie so spätnou väzbou** – skupiny procesov rozdelené podľa priorít, novo vzniknutý proces je vo fronte s najvyššou prioritou, postupne klesá, nakoniec je plánovaný Round-Robin v najnižšej úrovni.
  - o **modifikácia**: proces zaradení do rady na základe statickej priority, dynamická priorita sa zvyšuje, pokiaľ veľa čaká na I/O, alebo naopak znižuje, pokiaľ spotrebovávajú veľa času CPU. Cieľ: rýchla reakcia interaktívnych procesov.

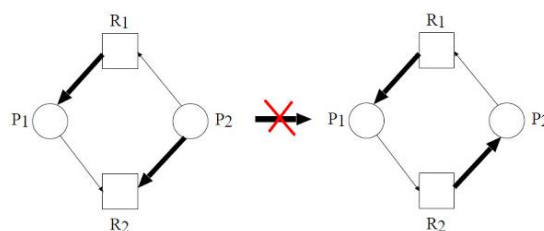
## 12. Kroky systému pri volaní `open()` [pr 4 , sl 53]

- Vyhodnotí sa cesta k otváranému súboru, posúdi existencia a prípustné práva a načíta sa číslo i-uzlu
- V tabuľke v-uzlu sa alokuje nová položka, od nej sa vloží i-uzol a rozšíri sa o niektoré ďalšie údaje vrátane počtu odkazov na daný v-uzol (aktuálne 1)
- Alokuje sa nová položka v tabuľke otváraných súborov a naplní sa odkazom na v-uzol a ďalšími informáciami. Hlavne:
  - o pozícia v súbore
  - o počet odkazov
  - o režim otvárania
- Alokuje sa prvá voľná položka v tabuľke deskriptora, naplní sa odkazom na položku tabuľky otvorených súborov
- Vráti index alokovaného deskriptora, alebo -1 pri chybe.

## 13. Deadlock

- Uviaznutie pri prístupe ku zdrojom s obmedzeným prístupom označujeme situáciu, keď každý proces z nejakej neprázdnej množiny procesov je pozastavený a čaká na uvoľnenie nejakého zdroja s výlučným obmedzeným prístupom vlastneného nejakým procesom z danej množiny, ktorý môže tento zdroj uvoľniť.
- Coffmanove podmienky:
  - o Vzájomné vylúčenie pri používaní prostriedkov
  - o Vlastníctvo aspoň jedného zdroja, pozastavenie a čakanie na ďalšie
  - o Prostriedky vrátia proces, ktorý je vlastný a to po dokončení ich využitia.
  - o Cyklická závislosť na sebe čakajúcich procesov
- Riešenie:
  - o Prevencia uviaznutia
  - o Vyhýbanie sa uviaznutiu
  - o Detekcia a zotavenie
  - o Overenie, že uviaznutie nemôže nastať

- Prevencia uviaznutia:
  - Zrušíme platnosť niektorých podmienok nutných pre uviaznutie
  - Nepoužívať zdieľané prostriedky, alebo používať zdieľané prostriedky, ktoré umožňujú (skutočne aktuálny) zdieľaný prístup a u ktorých nie je nutné vzájomné vylúčenie procesov
  - Proces môže žiadať o prostriedky iba, pokiaľ žiadne nemá
  - Pokiaľ proces žiada o prostriedky, ktoré nemôže momentálne získať, je pozastavený, všetky prostriedky sú mu odobrané a proces je zrušený, alebo sa čaká, až mu môžu byť všetky potrebné prostriedky pridelené
  - Prostriedky sú číslované a je možné je získavať iba od najnižších čísel k vyšším, alebo iným spôsobom vylučujúcim cyklickú závislosť na sebe čakajúcich procesov
- Vyhýbanie sa uviaznutiu:
  - Procesy vopred deklarujú určité informácie o spôsobe, aké budú využívať zdroje. V najjednoduchšom prípade sa jedná o maximálny počet súčasne požadovaných zdrojov jednotlivých typov.
  - Vopred známe informácie o možných požiadavkách jednotlivých procesov o aktuálnom stave prideľovania využijú k rozhodovaniu o tom, ktoré požiadavky môžu byť uspokojené (a ktoré musia počkať) tak, aby nemohla vzniknúť cyklická závislosť na seba čakajúcich procesov aj v najhoršej možnej situácii, ktorá by mohla v budúcnosti vzniknúť pri deklarovanom správaní procesu
  - Existuje rada algoritmov pre vyhýbanie sa uviaznutiu, napríklad algoritmus založený na grafe alokácie zdrojov pre systémy s jednou inštanciou každého zdroja. Vytvára sa graf vzťahu s 2 typmi uzla (P- procesy, R- zdroje) a tromi typmi hrán ( $R \rightarrow P$ ) – zdroj je vlastnený procesom, ( $P \Rightarrow R$ ) – proces žiada o zdroj, ( $P \rightarrow R$ ) proces môže požiadať o zdroj
  - Zdroj je pridelený iba vtedy, ak nehrozí vznik cyklickej závislosti čakajúcich procesov, čo by sa prejavilo cyklom v grafe pri zámene hrany žiadosti za hranu vlastníctva



Zobecnením je tzv. Bankerov algoritmus

- Detekcia uviaznutia a zotavenia
  - Uviaznutie môže vzniknúť, periodicky sa pri tom detekuje, či k uviaznutiu nedošlo, ak áno, prevedie sa zotavenie.
  - Detekcia uviaznutia: Graf vlastníctva zdrojov a čakania na zdroje procesy (podobný ako graf alokácie zdrojov, ale bez hrán vyjadrujúcich možnosť požiadať o zdroj), cyklus indikuje uviaznutie
  - Zotavenie z uviaznutia:
    - Odobranie zdrojov aspoň niektorými pozastavenými procesmi, ich pridelenie ostatným a neskôršie umožnenie získať všetky potrebné zdroje a pokračovať (prípadne ich reštart či ukončenie)
    - V každom prípade anulácia nedokončených operácií (rollback), alebo nutnosť akceptácie možných nekonzistencií

#### 14. Operačný Systém, Jadro, Mikrojadro [pr 1, sl 9-16]

##### - OS:

- kolekcia programov vytvárajúca spojovaciu vrstvu medzi (prípadne virtualizovaným) HW a užívateľmi a ich aplikáciami
- Ciele:
  - Maximálne využitie zdrojov (v minulosti, drahé počítače x lacná pracovná sila)
  - Jednoduchosť použitia (v súčasnosti, lacné počítače x drahá pracovná sila)
- Role:
  - Tvorca virtuálneho počítača – abstrakcie, štandardné rozhranie
  - Správca zdrojov – bezpečnosť a efektivita zdieľania

##### - Jadro

- Jadro OS je najnižšia a najzákladnejšia časť OS
- Najnižšia správa prostriedkov a tvorba prostredí pre zvyšok OS a užívateľ. Aplikácie
- Reaktívny program, zavádza sa ako prvý pri zapnutí a beží celú dobu
- Nadväzuje priamo na HW, typicky ho pre užívateľa zapuzdruje
- Obvykle beží v privilegovanom režime
- Zaisťuje základnú správu prostriedkov a tvorbu prostredí

##### - Rozlišujeme dva typy rozhraní:

- Kernel Interface: priame volanie jadra špecializovanou inštrukciou
- Library Interface: volanie funkcie zo systémových knižníc, ktoré môžu (ale nemusia) viesť na volanie služieb jadra

##### - Typy jadier:

- **Monolitické jadro:** všetok jeho kód beží na rovnakom (jadrovom) pamäťovom priestore, ktorý sa označuje ako kernel space. Tým sa líši od tzv. mikrojadra, ktoré väčšinu tradičných činností monolitického jadra, ako je napríklad správa súborových systémov, implementuje v procesoch, ktoré bežia v užívateľskom pamäťovom priestore. Aj napriek tomu, že sú monolitické jadrá písané tak, aby bola činnosť jednotlivých subsystémov oddelená, sú jednotlivé časti veľmi silne prepojené. Z dôvodu, že zdieľajú rovnaký pamäťový priestor, môže chyba v jednom subsystéme zablokovat iný, alebo dokonca zhodiť celé jadro. Na druhú stranu, pokiaľ je dôraz daný na správnu implementáciu jednotlivých častí, je monolitické jadro veľmi efektívne.
- **Mikrojadro:** je veľmi malé a obsahuje len najzákladnejšie časti, čím sa minimalizuje objem bežiaceho kódu v privilegovanom systéme. Ostatné potrebné časti jadra sú riešené v užívateľskom priestore ako bežné procesy (resp. démoni, u mikrojadier sa označujú servery), napríklad správa súborového systému, ovládače zariadení, podpora protokolov pre počítačové siete a ďalšie.
  - Výhody: flexibilita, zabezpečenie
  - Nevýhody: vyššia réžia
- **Hybridné jadrá:** Mikrojadrá rozšírené o kód, ktorý by mohol byť implementovaný vo forme serveru, ale je za účelom menšej réžie tesnejšie previazaný s mikrojadrom a beží v jeho režime.
- **Exojadrá:** Experimentálne jadrá poskytujúce veľmi nízke rozhranie zamerané hlavne na bezpečnostné zdieľanie prostriedkov. Doplnené o knižnice implementujúce služby inak ponúkané jadrom. Zatiaľ sa nepresadili v praxi.

## 15. Žurnálovanie [pr 4, sl 17-20]

- Žurnál slúži pre záznam modifikovaných metadát (dát) pred ich zápisom na disk. Implementovaný ako cyklicky prepisovaný buffer v špeciálnej oblasti disku. Operácie pokryté žurnalovaním sú atomické: vytvárajú transakcie – buď uspejú všetky dielčie kroky, alebo žiadny
- Systémy so žurnálom: Ext3, Ext4, NTFS
- Umožňuje spoľahlivejší a rýchlejší návrat do konzistentných stavov po chybách.
- Dáta zvyčajne žurnálové nie sú – veľká réžia
  - o Kompromis: predradenie zápisu dát na disk pred zápisom metadát do žurnálu
- Implementácia:
  - o Implementácia na základe dokončenia transakcií – REDO:
    - Sekvencia dielčích procesov operácií sa uloží do žurnálu medzi značkami štartu a konca transakcie. Potom sa na disku prevedú dielčie operácie. Ak uspejú všetky, transakcie sa zo žurnálu uvoľnia. Ak nie, dokončia sa všetky transakcie, ktoré sú v žurnále zapísané celé.
  - o Implementácia na základe anulácie transakcií – UNDO:
    - Záznam dielčích procesov do žurnálu a na disk sa prekladá. Ak prebehne celá transakcia, zo žurnálu sa uvoľní. Pri chybe sa eliminujú nedokončené transakcie.
  - o UNDO+REDO (NTFS)
  - o Implementácia musí zaistiť správne poradie zápisu operácií, ktoré ovplyvňujú plánovanie diskových operácií v OS, alebo ich preusporiadanie na disku.
- Alternatívy:
  - o *Copy-on-Write*
    - Najprv zapisuje nové metadáta na disk, potom ich sprístupní: Zmeny robí hierarchicky v súlade s hierarchickým popisom obsahu disku (ide o vyhľadávajúci, nie adresovací strom). Vytvorí sa kópia meneného uzla -> upraví ju -> vytvorí kópiu uzla nadriadeného -> upraví ju tak, aby ukazovala na uzol vytvorený v predchádzajúcom kroku. Na najvyššej úrovni sa udržuje niekoľko verzií koreňového záznamu so zabezpečovacím kódom a časovými razítkami. Ponúka sa pre implementáciu: snímky súborového kódu a klony SS – vznikajú čiastočne zdieľané stromové štruktúry popisujúce rôzne verzie obsahu SS

## 16. Súbor popísaný B+ stromom [pr4 , sl 27-29]

- V listoch sú 3 odkazy, jeden z nich je potrebný pre prevádzanie do zoznamu, ostatné 2 odkazy na dáta a celkom sú potrebné 3 listy.
- 3 listy sa dajú adresovať z jedného koreňa
- B+ stromy sa ďalej používajú k popisu
  - o obsahov adresárov
  - o voľného priestoru na disku.
- Vkladajú sa na listovej úrovni. Ruší sa od nej.

## 17. Alokačný blok, Diskový sektor, Extent [pr 4, sl 3,11,29]

- **Diskový sektor:** najmenšia jednotka, ktorú disk umožňuje načítať/zapísať
- **Veľkosť sektoru:** typicky 512B u nových diskov 4096B (s emuláciou 512B)
- **Adresácia sektorov:**
  - o CHS = Cylinder, Head (typicky 1-6), Sector
  - o LBA = Linear Block Address (číslo 0 až N)

- **Alokačný blok:** skupina pevného počtu sektorov, typicky  $2^N$  pre nejaké  $N$ , nasledujúcich logicky (tj. V súbore) aj fyzicky (tj. Na disku) za sebou, ktorí je najmenšou jednotkou diskového priestoru s ktorou vie OS (FS) pracovať. (najmenšia ktorá sa dá načítať/zapísať)
- **Extent:** postupnosti premenného počtu blokov za sebou idúcich logicky v súbore a uložených fyzicky za sebou na disku. Zrýchľujú prácu s veľkými súbormi (menšie indexovacie štruktúry, menší objem metadat), naopak pre malé súbory to môže znamenať zbytočne veľkú režiu

#### 18. Preklad logickej adresy na fyzickú cez 2-úrovňovú tabuľku stránok

- LA sa rozloží na prefix  $p$  zložený z odkazov  $p_1$  do adresára stránok a z odkazu  $p_2$  do konkrétnej tabuľky stránok. Teda  $p = p_1 * p_2$
- Test, či TLB obsahuje dvojicu  $(p, f)$ , vyhľadáva sa asociatívne podľa  $p$  (je možné čiastočne asociatívne vyhľadávanie), ak je položka  $(p, f)$  nájdená, je výsledná adresa  $(f, d)$
- Prístup k položke na indexu  $p_1$  v adresári stránok, ktorý je v RAM na adrese uloženej v špeciálnom registre MMU (u intelu je to CR3)
- Ak má vyššie uvedená položka nastavený príznak neplatnosti, nastáva výpadok v adresári stránok
- Ak tomu tak nie je, potom sa z uvedenej položky získava bazová adresa dielčej tabuľky stránok taktiež uloženej v RAM. Použije sa položka s indexom  $p_2$  tejto tabuľky.
- Ak obsahuje položka na indexu  $p_2$  v dielčej tabuľke stránok príznak neplatnosti, nastáva výpadok stránky. Inak sa z položky zmienenej vyššie získa číslo rámca  $f$  a výsledná adresa  $(f, d)$

#### 19. Inverzia priorít [pr 5, sl 26,27]

- **Problémy inverzie priorít:**
  - o Nízko-málo prioritný proces si naalokuje nejaký zdroj, viac prioritné procesy ho predbiehajú a nemôže dokončiť prácu s týmto zdrojom.
  - o Časom tento zdroj môže potrebovať viac prioritnejšie procesy, sú nutne zablokované a musia čakať na nízko-málo prioritný proces.
  - o Ak sú v systéme v tomto momente stredne prioritné procesy, ktoré nepotrebujú daný zdroj, tak pobežia a budú ďalej predbiehať nízko-málo prioritný proces
  - o Týmto spôsobom uvedené stredné a nízko-málo prioritné procesy získavajú efektívne vyššiu prioritu
- Inverzia priorít nemusí, ale môže, vadiť; môže zvyšovať odozvu systému a spôsobiť aj vážnejšie problémy, najmä keď sú blokované nejaké kritické procesy reálneho času (ovládanie hardware apod.).
- **Možnosti riešenia inverzie priorít:**
  - o **Priority ceiling:** procesy v kritickej sekcii získavajú najvyššiu prioritu.
  - o **Priority inheritance:** proces v kritickej sekcii, ktorý blokuje viac prioritné procesy, dedia (počas prebiehania v kritickej sekcii) prioritu čakajúceho procesu s najväčšou prioritou.
  - o **Zákaz prerušenia po dobu behu v kritickej sekcii** (v jednoprocessorovom systéme): proces v podstate získa vyššiu prioritu než všetky ostatné.
- **Ďalší výraznou komplikáciou plánovania predstavujú:**
  - o **Viacprocesorové systémy** - nutnosť vyvažovania výkonu, rešpektovanie obsahu cache procesorov, príp. lokalitu pamäte.
  - o **Hard real-time systémy** - nutnosť zaistiť garantovanú odozvu niektorých akcií.

#### 20. Nežiadúce účinky využitia operácie spinlock

- Pri dlhom čakaní môže blokovať prostriedky iným procesom, ktoré potom nemôžu pokračovať. Zaťažuje procesor, pretože proces stále beží v cykle.

#### 21. NTFS, odlišnosti rôzne dlhých súborov, 3 varianty reprezentácie a adresovania [ pr 4; sl 31 ; ]

- MFT (Master File Table)
  - o aspoň jeden riadok pre každý súbor
  - o malé súbory sú celé uložené v MFT (aj s obsahom)
  - o stredné – na riadku príslušného súboru sú uložené odkazy na extenty
  - o veľké – cez 2 riadky sú odkazy na pomocné odkazy, v ktorých sú uložené odkazy na extenty (adresovanie podobné ako pri B+ stromoch)

- Extent
  - o postupnosť premeneného počtu alokačných blokov logicky idúcich za sebou v súbore a uložených fyzicky na disku
- Podpora žurnálovania (REDO+UNDO)

## ÚLOHY / PRÍKLADY

### 1. Monitory pomocí semaforů [pr 6; sl 12,13,16]

**Wait - Notify** \*\*\*\*\*

```
struct sMonitor tMonitor;
void enter (tMonitor *m){
    lock(m->mutex);
}

void wait (tMonitor *m){
    m->condCount++;
    unlock(m->mutex);
    lock(m->condition);
    lock(m->mutex);
}

void notify (tMonitor *m){
    if (m->condCount > 0){
        unlock(m->condition);
        m->condCount--;
    }
}

void leave (tMonitor *m){
    unlock(m->mutex);
}
```

**Wait - Signal** \*\*\*\*\*

```
typedef struct{
    semaphore mutex, condition, priority;
    int prioCount, condCount;
}tMonitor;
void enter (tMonitor *m){
    lock(m->mutex);
}

void wait (tMonitor *m){
    m->condCount++;
    if(m->prioCount > 0)
        unlock(m->priority);
    else
        unlock(m->mutex);
    lock(m->condition);
    m->condCount--;
}

void signal (tMonitor *m){
    if (m->condCount>0) {
        m->prioCount++;
        unlock(m->condition);
        lock(m->priority);
        m->prioCount--;
    }
}

void leave (tMonitor *m){
    if(m->prioCount > 0)
        unlock(m->priority);
    else
        unlock(m->mutex);
}
```

## 2. Implementace lock s využitím spinlocku (Swap) [pr 6; sl 9,12,13]

```
struct sSemaphore{
    int value;
    bool locked;
    queue *q;};
}
void init(struct sSemaphore *s, unsigned value){
    s->value = value;
    s->locked = false;
}
void lock(struct sSemaphore *s){
    bool key = true;
    while(key == true){
        swap(&(s->locked), &key);
    }
    s->value--;
    if (s->value < 0){
        process_t *C=get(ready_q) ;
        append(s->q,C);
        s->locked = true
        switch();
    } else {
        s->locked = false;
    }
}
```

## 3. Implementace lock včetně spinlocku (Test&Set) [pr 6; sl 13]

```
struct sSemaphore{
    int value;
    queue *q;
    bool locked;
}
void init(struct sSemaphore *s, unsigned value){
    s->value = value;
    s->locked = false;
}
void lock(struct sSemaphore *s){
    while(Test&Set(s->locked));
    s->value--;
    if (s->value < 0){
        process_t *C=get(ready_q);
        append(s->q,C);
        s->locked = false;
        switch();
    }else
        s->locked = false;
}
```

## 4. Semafor, funkce unlock s využitím spinlock [př 6, sl 14]

Test&Set

```
struct sSemaphore{
    int value;
    queue *q;
    bool locked;
}
void unlock(sSemaphore *s){
    while(Test&Set(s->locked));
    s->value++;
    if (s->value <= 0) {
        queue *p = get(s->q);
        append(ready_queue, p);
        s->locked = false;
    }
    else
```

```

        s->locked = false;
    }
    Swap
    struct sSemaphore{
        int value;
        queue *q;
        bool locked = false;
    }
    void unlock(sSemaphore *s){
        bool key = true;
        while(key){
            swap(&(s->locked), &key);
            s->value++;
            if (s->value <= 0) {
                queue *p = get(s->q);
                append(ready_queue, p);
                s->locked = false;
            }
            else
                s->locked = false;
        }
    }

```

## 5. Vyhľadávání v tabulce stránek [pr 7; sl 18,19]

### Invertovaná tabulka

```

struct inv_tab_item{
    pid_t pid;
    page_num_t pg;
}
page_num_t page_to_frame(...){
    for (page_num_t i = 0; i < max_frame_num; i++){//page_num_t i = 0; i<...
        if (inv_tab[i].pid == pid && inv_tab[i].pg == pg){
            return i;
        }
    }
    return PAGE_FAULT;
}

```

### Invertovaná tabulka s hashováním

```

struct inv_tb_item{
    pid_t pid;
    page_num_t pg;
    page_num_t next;    // Tohle místo struct
}
page_num_t page_to_frame(...){
    page_num_t fr = hash(pg, pid);
    while (inv_tb[fr].pid != pid || inv_tb[fr].pg != pg){
        if (inv_tb[fr].next == NEGATIVE)
            return NEGATIVE;
        else
            fr = inv_tb[fr].next;
    }
    return fr;
}

```

### Hashovací tabulka

```

struct hash_tab_item{
    pid_t pid;                // PID iba pri zdieľanej hash. tabulke
    page_num_t pg;
    page_num_t fr;
    struct hash_tab_item* next;
}
page_num_t page_to_frame(struct hash_tab_item* hash_tab, pid_t pid, page_num_t
page_num){
    page_num_t index = hash(pid,page_num);
    struct hash_tab_item* itr = &hash_tab[index];
}

```

```

        while (itr)
        {
            if (itr->pid == pid && itr->pg == page_num)
                return itr->fr;
            itr = itr->next;
        }
        return NEGATIVE;
    }
}

```

#### Hashovací tabulka (regiony)

```

struct hash_tab_item {
    page_num_t pg;
    reg_t reg;
    page_num_t fr;
    struct hash_tab_item* next;
}

page_num_t page_to_frame(struct hash_tab_item* hash_tab, pid_t pid, reg_t
reg, page_num_t page_num) {
    reg_t glob_reg = loc_to_global_reg(pid, reg); // z lokalneho regionu
na globalny
    page_num_t index = hashPg(pg, glob_reg);
    struct hash_tab_item* itr = hash_tab[index];
    while (itr) {
        //if (*itr->reg == reg && */itr->pg == page_num)
        if (itr->pg == page_num)
            return itr->fr;
        itr = itr->next;
    }
    return PAGE_FAULT;
}

```

#### Deklaracia hashPg

```
int hashPg(page_num_t pg, reg_t global_reg) { ... }
```

### 6. Synchronizace tří procesů

```

Init:      init(s1, 0);
           init(s2, 0);
           init(s3, 0);

p1:        printf("p1: krok 1\n");
           unlock(s2);
           lock(s1);

p2:        printf("p1: krok 2\n");
           lock(s2);
           printf("p2: krok 1\n");
           unlock(s3);
           lock(s2);
           printf("p2: krok 2\n");
           unlock(s1);

p3:        lock(s3);
           printf("p3: krok 1\n");
           printf("p3: krok 2\n");
           unlock(s2);

```

## Vypracované zadania

1. Zadání: Jaký je maximální počet výpadků stránek v systému se stránkami o velikosti 4 kB, 4-úrovňovou tabulkou stránek, u které pouze dílčí tabulka nejvyšší úrovně je chráněna proti výpadku, při provádění předem načtené instrukce o délce 4 B, která přesouvá 8 kB z jedné adresy paměti na jinou?

Řešení: (takhle bych to psal na písemku)

- instrukce se může rozdělit do dvou stránek, ale je předem načtená (0)
- data se mohou rozdělit až do tří stránek, čtení i zápis
- výpadek může nastat při samotném čtení dat ( $3 * 1$ )
- výpadek může nastat při cestě ke stránkám, avšak dílčí tab. je chráněná ( $N - 1 \rightarrow 4 - 1$ )

- cesta ke stránkám se může provést 2x, kvůli rozděleným datům do tří stránek ( $2 * (N - 1) \rightarrow 2 * (4 - 1)$ )
- to vše se může opakovat 2x kvůli čtení a zápisu ( $2 * (3 + 2 * (4 - 1))$ )
- $2 * (3 + 2 * (4 - 1)) = 18$

**2. Zadání:** Architektura se šířkou slova 256 bitů používá stránkovaný přístup do paměti, velikost stránky je  $2^{32}$  B, hierarchická tabulka stránek a dílčí tabulky mají velikost vždy právě 1 stránky a položky o velikosti 256 b. Kolika úrovněovou tabulku stránek může systém používat, když stránky jsou plně využitelné, v tabulkách nejsou nevyužité řádky a maximálně horních 16b adresy není použito pro adresování?

Řešení: (takhle bych to psal na písemku)

- 32 b pro posuv
- $256 - 32 = 224$  b pro stránkování
- $256b = 2^8 B$
- $2^{32} / 2^8 = 2^{24}$  řádků v dílčí tab. str.
- 27 b pro adresaci řádků
- $224 / 27 = 8,296 \rightarrow 8$  odkazů
- kontrola:  $8 * 27 = 216$ ;  $224 - 216 = 8 \rightarrow 8 < 16$  (správně)
- wtf
- takže kolko úrovněovou?
- **8 odkazů  $\rightarrow$  8 úrovní**

**3. Zadání:** Jaký je maximální počet přístupů do paměti při použití 4-úrovněové tabulky stránek u instrukce o délce 4 B přistupující pro 4 B do paměti (čtení nebo zápis)? Jak se tento počet změní v případě přesunu v paměti? Jak se toto maximum změní, víme-li, že se jedná o instrukci, která v kódu lineárně následuje za aktuálně prováděnou instrukcí?

Řešení: (takhle bych to psal do písemky)

1. **modrá**

- instrukce se může rozdělit do dvou stránek
  - cesta ke stránkám ( $N = 4$ )
  - čtení stránky (1)
  - to vše 2x, kvůli rozdělení ( $2 * (N + 1)$ )
- data se mohou rozdělit do dvou stránek
  - cesta k datům ( $N = 4$ )
  - čtení stránky (1)
  - to vše 2x, kvůli rozdělení ( $2 * (N + 1)$ )
- $(2 * (4 + 1)) + (2 * (4 + 1)) = 20$

2. **červená**

- stejný postup jako v a), jen se přístup k datům zopakuje
- $2 * (2 * (4 + 1)) + (2 * (4 + 1)) = 30$

3. **zelená**

- instrukce se může rozdělit do dvou stránek, ale jedna stránka již je načtená z předešlé instrukce
  - čtení instrukce v načtené stránce (1)
  - čtení instrukce v nenačtené stránce (1)
  - cesta k nenačtené stránce přes úrovně ( $N = 4$ )
  - s daty je to totožné jako předešlé příklady
- vyberu variantu a);  $(2 * (4 + 1)) + (1 + 4 + 1) = 16$

**4. Zadání:** Předpokládejme systém s virtuální pamětí, ve které se používá lokální výměna stránek a ve kterém se při výběru odkládané stránky používá algoritmus LRU. Předpokládejme dále, že v tomto systému je spouštěn proces, který má napevno přiděleny 4 rámce, do nichž na začátku není namapována žádná stránka. Ke kolika výpadkům stránky dojde, pokud daný proces postupně přistoupí k následujícím stránkám: 1 2 3 4 4 1 5 2 2 3 5 1.

Řešení:

- rámec je prázdný, vejdu se do něj první 4 stránky (4 výpadky) (obsah rámce: 1 2 3 4; zbývá: 4 1 5 2 2 3 5 1)
- 4 již v rámci je (0 výpadků) (obsah rámce: 1 2 3 4; zbývá: 1 5 2 2 3 5 1)
- 1 již v rámci je (0 výpadků) (obsah rámce: 1 2 3 4; zbývá: 5 2 2 3 5 1)
- 5 není v rámci, dle algoritmu nahradíme za příslušnou hodnotu, v tomto případě za 2 (1 výpadek) (obsah rámce: 1 5 3 4; zbývá: 2 2 3 5 1)
- 2 byla v předešlém kroku vyhozena (riziko LRU algoritmu), nahradíme za 3 (1 výpadek) (obsah rámce: 1 5 2 4; zbývá: 2 3 5 1)
- 2 již v rámci (0 výpadků) (obsah rámce: 1 5 2 4; zbývá: 3 5 1)
- 3 není v rámci, nahradíme za 4 (1 výpadek) (obsah rámce: 1 5 2 3; zbývá: 5 1)
- 5 je v rámci (0 výpadků)
- 1 je v rámci (0 výpadků)
- $4 + 0 + 0 + 1 + 1 + 0 + 1 + 0 + 0 = 7$

### Alokační blok (Cluster)

Nejmenší logická část souborového systému. Cluster je vždy  $2^n$  sektorů následujících za sebou.

#### extent

- posloupnost proměnného počtu bloků uložených na disku fyzicky za sebou
- extent udává, kde na disku začíná a kolik bloků obsahuje
- snižuje se počet metadat, zrychluje se čtení velkých souborů
- používají se např v B+ stromech

#### žurnálování, žurnál ...

Slouží pro záznamy změn u metadat/dat před jejich zápisem na disk.

- používají je např ext3, ext4, ReiserFS, NTFS
- většina dat žurnálována není (příliš velká režie)

#### start systému:

- 1.BIOS
- 2.inicializace služeb jádra+jádro samotné
- 3.swapper
- 4.init

**možná implementace semaforu (je nutné zajistit aby semafor proběhl jako atomická instrukce) využije se vlastnosti spinlocku ...**

```
typedef Struct {
    int value;
    fronta_procesu *queue;
    bool lock;
} semaphore;
lock(S){
    while(testAndSet(S.lock)); // spinlock
    S.value--; //nevím proc
    if(S.hodnota < 0){
        C = get(ready_queue); // vytahnu z fronty procesu pripravenych bezet
        append(S.queue, C); // dam na forntu cekajicich procesu na semaforu
        S.lock = false; // vychzim z KS
        switch(); // vzdam se jadra
    }
    else{
        S.lock = false; // vychazim z KS
    }
}
```

**// jak by se zamykalo se spinlockem ...**

```
bool TestAndSet(bool &target) {
    bool rv = target;
    target = true;
    return rv;
}
```

Tento kód popisuje chování spinlocku, který je ovšem řešen hardwarově za účelem dosažení atomicity.

S.value--; je tam proto, že při zamčení semaforu se dekrementuje jeho hodnota.

#### Jak probíhá překlad adres při stránkování?

```
pg_num page; //cislo stranky
pg_num PageTable; //velikost tabulky
int page_item[PageTable];
fr_num frame; //cislo ramce
if (page<=PageTable)
{
    frame=page_item[base+page];
    physic_adress=frame+offset;
}
```

else SIGSEG;

### **úrovně běhu, co znamenají? které se využívají?**

0..halt

1..single user

6..reboot

2-5..víceuživatelské užití, grafický režim, síťový režim,

grafický+síťový,.. záleží na distribuci

změna telinit N (kde N je 0-6 viz výše)

### **Co je hw přerušení, co je řadič přerušení, jak lze zakázat**

přerušení, co je NMI, rozdělení na úrovně ...

HW přerušení je mechanismus, kterým HW zařízení oznamují jádru asynchronně vznik událostí, které je zapotřebí obsloužit.

Řadič přerušení je řadič, do kterého přicházejí žádosti o HW přerušení. Na PC se jmenuje APIC, kde každý procesor má vlastní lokální APIC.

Řadič může být naprogramován tak, aby maskoval určitá přerušení.

Obsluhu přerušení lze také zakázat na procesoru, případně čistě programově v jádře.

NMI (Non-maskable interrupt) je HW přerušení, které nelze maskovat na řadiči, ani zakázat jeho přerušení na procesoru.

Obsluha přerušení bývá rozdělena na 2 úrovně:

1) Zajišťuje minimální obsluhu HW a plánuje běh obsluhy 2. úrovně.

2) Postupně řeší zaznamenaná přerušení.

### **Vše, co víte o deadlocku (cca za 15b)**

Vzniká když více procesů chtějí dvě zařízení. A nastane situace že 1. proces si zabere jedno zařízení a 2. zařízení druhé a oba čekají na jejich druhé zařízení které je již zabrané.

řešení: výlučný přístup, postupné přidělování prostředků, odebrání zařízení po určité době

podmínky uváznutí:

vzájemné vyloučení při používání prostředků

vlastnictví alespoň jednoho zdroje a čekání na další

prostředky vrací pouze proces po dokončení jejich využití

cyklická závislost na sebe čekajících procesů

prevence:

-zruší se některá z platnosti podm. uváznutí

1.u prostředků, které umožňují (současný) sdílený přístup, nejsou zámky zapotřebí

2.proces může žádat o prostředky pouze pokud žádné nevlastní

3.pokud proces požádá o prostředky, které nemůže momentálně získat, je pozastaven, všechny prostředky jsou mu odebrány a čeká se, až mu mohou být všechny potřebné prostředky přiděleny

4.prostředky jsou očíslovány a je možné je získávat pouze od nejnižších čísel k vyšším

vyhýbání:

- procesy před spuštěním deklarují určité informace o způsobu, jakým budou využívat zdroje: v

nejjednodušším případě se jedná o maximální počet současně požadovaných zdrojů jednotlivých typu.

- předem známé informace o možných požadavcích jednotlivých procesu a o aktuálním stavu

přidělování se využijí k rozhodování o tom, které požadavky mohou být uspokojeny (a které musí

počkat) tak, aby nevzniklo cyklické čekání.

zotavení a uváznutí:

-ukončení všech nebo některých zablokovaných procesu,

-odebrání zdroje některým procesum, anulace jejich nedokončených operací

(rollback) a pozdeji restart.

este tam napríklad mozes dodat, ze ma zurnalovanie....

Vyuziva istu modifikáciu B+ stromov, nazývaných tiež H-stromy. Na rozdiel od Unixu metadata obsahujú aj názov suboru. Ak sa špecifikácia a prístupové práva nevojdu do vyhradeného priestoru, alokuje si ďalšie riadky. Každý subor má defaultne "nachystaný" jeden riadok v tabuľke. Oblasť pre data obsahuje adresu začiatku extendu a to ako aj logickú tak aj fyzickú a veľkosť. Opat, ak je počet riadkov adresuje sa priamo, ak nie alokujú sa ďalšie riadky na ktoré sa odkazuje, podobne ako v i-uzloch.

?+ to, že je to proprietárny FS od Microsoftu (docela podstatná informace)